Architecture, Technical Design & ADRs

VERSION 1.1 | 06 NOVEMBER 2026

400 COFFEE

Architecture, Technical Design & ADRs

Logical architecture, domain model, interfaces, security and decision records

Document status: Professional working draft for stakeholder review. Open decisions and assumptions are explicitly identified.

Contents

1. Architecture summary

2. Context and container view

3. Domain modules

4. Core data design

5. Payment state and invariants

6. API and integration conventions

7. ADR register

8. Security, privacy and operations

9. Deployment and environments

1. Architecture summary

Recommended shape A modular monolith with clear domain boundaries, backed by PostgreSQL. This minimizes operational overhead while preserving seams for later extraction if scale or team ownership justifies it.

Client applications React + TypeScript progressive web apps for cashier POS, kitchen display and customer QR. Next.js + TypeScript for owner and platform administration surfaces where server rendering and reporting navigation are useful.

Backend Java 21 and Spring Boot REST APIs. PostgreSQL as system of record. Redis introduced only where justified for caching, rate limits, short-lived coordination or queue support. Server-sent events or WebSockets for operational updates.

Infrastructure Containerized deployment, managed PostgreSQL, object storage for exports/assets, centralized logs/metrics/traces, secrets manager, automated backups and CI/CD. Select cloud provider/region after cost, residency and support review.

2. Context and container view

Actors Customer, cashier, beverage station, food station, owner/manager, SaaS platform operator.

Containers Customer QR PWA; Staff POS PWA; KDS PWA; Owner/Admin web app; Spring Boot API; PostgreSQL; optional Redis; event/outbox worker; notification adapter; external UPI/bank channel.

Trust boundaries Public QR endpoints are rate-limited and token-scoped. Staff/admin endpoints require authenticated principals. Tenant/outlet scope is derived from server-side identity and authorization, never trusted from request body.

Data flow Clients call REST APIs for commands and authoritative reads. Backend commits business state and outbox event transactionally. Realtime channel publishes hints. Clients reconnect and refetch queue/state from REST.

3. Domain modules

Identity and access Users, credentials, sessions, roles, permissions, tenant/outlet membership.

Tenant and subscription Tenant, outlet, plan, subscription, entitlement, usage and platform administration.

Catalog Category, product, price, tax/charge configuration, modifier group, modifier, availability and station routing.

Ordering Order, order item, order item modifier, order source, table, lifecycle, customer session and secure tracking token.

Kitchen Kitchen station, station ticket, ticket item, preparation status, timer and event recovery.

Payments and billing Payment attempt, payment ledger entry, verification, reference, refund, bill, shift and reconciliation.

Notifications and analytics Notification, delivery attempt, metric definitions, report snapshot and audit event.

Offline sync Device, local operation identifier, sync event, idempotency key and conflict record.

4. Core data design

Tenant ownership Every tenant-owned record carries tenant_id; outlet-scoped records also carry outlet_id. Composite constraints and query filters reinforce scope. Tenant ID is not accepted as an authorization fact from clients.

Order snapshot Persist item name, SKU, unit price, modifier selections, tax/charge basis and totals as immutable order-time snapshots. Catalog edits must not rewrite historical orders.

Split tickets Order is parent aggregate; station tickets are child execution records. Store station routing snapshot at acceptance. Parent readiness derives from all required ticket items.

Payment separation PaymentAttempt captures a collection attempt; PaymentLedgerEntry captures money movement/verification/refund; PaymentVerification records who checked what evidence and when. Order status and payment status remain separate.

Public QR security QR identifier resolves outlet/table context but does not grant staff privileges. Tracking token is random, high entropy, scoped to one order and revocable/expiring. Avoid sequential order IDs as access tokens.

Audit and events Use append-only audit events for critical operations. Transactional outbox ensures state and event publication intent commit together. Events are delivery hints; REST remains authoritative.

5. Payment state and invariants

States NOT_STARTED, CUSTOMER_DECLARED, PENDING_VERIFICATION, PARTIALLY_PAID, PAID, FAILED, OVERPAID, REFUND_PENDING, REFUNDED, RECONCILIATION_REQUIRED.

Invariant 1 Customer declaration, UPI intent, screenshot or customer-provided reference never independently marks an order PAID.

Invariant 2 Only authorized server-side verification can add confirmed collection to the ledger.

Invariant 3 Verified allocations cannot exceed the amount received; duplicate merchant references are rejected or escalated.

Invariant 4 Refunds are separate ledger movements and never erase original collections.

Invariant 5 Gross order value, confirmed collections, refunds, outstanding receivables and net collections are distinct metrics.

6. API and integration conventions

Base path /api/v1. Use JSON, UUID identifiers, ISO-8601 UTC timestamps and decimal strings for monetary amounts.

Headers Authorization; Idempotency-Key on retry-sensitive writes; X-Request-Id for correlation.

Authorization Server derives tenant/outlet permissions from authenticated principal. Public endpoints use narrowly scoped QR/tracking tokens.

Errors Stable machine-readable error codes, safe user messages, field validation details and request ID. Never leak cross-tenant entity existence.

Commands Create/update/cancel/verify commands validate state transitions server-side and return authoritative resource representation.

Realtime Events include order.created, order.accepted, order.released, order.cancelled, kitchen.ticket.started, kitchen.ticket.ready, payment.declared, payment.verified and refund.created. Reconnect requires queue/state refetch.

Contract Publish OpenAPI; version endpoints; generate client types; run consumer/provider contract tests.

7. ADR register

ADR-001 Modular monolith Decision: modular monolith. Rationale: one pilot outlet and 200–500 daily orders do not justify microservice operational complexity. Revisit when independent scaling/team ownership or fault isolation becomes measurable.

ADR-002 Static UPI QR pilot Decision: manual merchant-side verification. Rationale: static QR cannot reliably bind a bank credit to an order. Revisit only with documented provider/bank integration and reconciliation support.

ADR-003 Cashier-led order release Decision: cashier is the control point for customer QR orders by default; configurable auto-release may be enabled by outlet policy. Policy is snapshotted per order.

ADR-004 Split kitchen tickets Decision: one parent order, station-specific tickets. Rationale: independent beverage/food preparation with aggregate readiness.

ADR-005 REST authoritative, realtime advisory Decision: REST APIs own truth; realtime events prompt refresh. Rationale: recoverability across disconnects and duplicate/missed messages.

ADR-006 Tenant isolation Decision: tenant_id plus outlet scope enforced at service/repository boundaries, supported by database constraints and automated negative tests. Consider stronger DB/schema isolation only when risk/compliance warrants.

ADR-007 PWA-first hardware Decision: browser-based tablet workflows initially. Rationale: flexible hardware procurement and simplified updates; validate printer/peripheral support on actual devices.

8. Security, privacy and operations

Security controls RBAC; tenant-aware authorization; MFA for privileged roles where feasible; secure cookies/token rotation; TLS; secrets manager; encryption at rest; dependency and container scanning; rate limiting; audit trails; backup restore drills.

Privacy controls Purpose limitation; minimal customer fields; retention schedule; deletion/export process; processor/subprocessor register; access logging; breach response; privacy notice and consent where required.

Observability Structured logs with tenant-safe identifiers; metrics for order creation, ticket latency, payment exceptions, queue depth, sync failures, API latency and availability; traces across command-to-event flows.

Recovery RPO/RTO targets are planning assumptions. Test restore, point-in-time recovery, queue rebuild and device reconnect. Do not claim offline customer ordering in V1.

Performance Validate p95 targets under 3× measured peak. Measure cold start, reconnect storm, ticket fanout and report queries separately.

9. Deployment and environments

Environments Local development, shared integration/test, staging/UAT and production. Isolate secrets and tenant data; production access controlled and audited.

CI/CD Build, unit tests, static analysis, dependency scan, container scan, migration validation, integration tests, deployment approval and smoke tests.

Database changes Versioned migrations, backward-compatible expand/contract where possible, backups before risky migrations, tested rollback/forward-fix procedures.

Runbooks Incident triage, payment mismatch, failed notification, KDS reconnect, device replacement, shift reconciliation, restore, tenant suspension and data export.

VERSION 1.1 — CUSTOMER EXPERIENCE ADDENDUM

Version 1.1 addendum — 06 November 2026. This addendum supersedes any earlier customer-facing UX description that implied a conventional QR menu/dashboard as the primary customer experience. The immersive Virtual Café is the approved customer-side direction. Existing staff POS, KDS, owner and platform-admin interfaces remain separate workstreams and are not redesigned by this customer-side change.

2.1 Customer experience runtime architecture

Keep the recommended modular-monolith backend and PostgreSQL. Customer experience is a separately deployable React + TypeScript PWA (or Next.js shell) with a dedicated scene layer. Use Three.js / React Three Fiber only if the selected team can meet mobile performance and accessibility gates; otherwise use a pre-rendered or CSS/WebGL hybrid with the same UX contract. Avoid coupling domain workflows to 3D scene state. Scene/dialogue components call typed customer-journey application services; the domain state machine remains authoritative on the server.

2.2 Front-end module boundaries

Suggested modules: customer-shell (routing, responsive layout, session); café-scene (camera, lighting, environment, manager rig and fallback); manager-dialogue (contextual scripts and accessibility); diary-menu (categories, product pages, search/filter);

catalog-client (API/cache); product-customizer (variants/modifiers/instructions); basket (local draft and server price refresh); customer-identity (name/mobile/consent); order-review; checkout/payment-adapter; bill-lookup; order-confirmation/status; notification-status; telemetry/feature-flags. Keep each module testable and avoid a single giant page component.

2.3 Customer order and payment state model

Order lifecycle (illustrative states; backend is authoritative): DRAFT → SUBMITTED → AWAITING_CASHIER_ACCEPTANCE (if policy requires) → ACCEPTED → IN_PREPARATION → READY → COMPLETED; cancellation/rejection/expiry are controlled terminal or branch states with actor/reason. Payment lifecycle remains separate: NOT_STARTED, PENDING, CUSTOMER_ACTION_REQUIRED, CUSTOMER_DECLARED, PENDING_VERIFICATION, PARTIALLY_PAID, PAID, FAILED, EXPIRED, REFUND_PENDING, REFUNDED, OVERPAID, RECONCILIATION_REQUIRED. Pay-at-counter order may be SUBMITTED with payment NOT_STARTED/DUE. Never infer PAID from order state.

2.4 API and integration contracts

Customer APIs (names are illustrative; finalize in OpenAPI): GET /public/outlets/{slug}/experience-config; GET /public/outlets/{slug}/menu; POST /customer/sessions; POST /customer/orders/quote; POST /customer/orders (idempotency key); GET /customer/orders/{publicRef}; POST /customer/orders/{publicRef}/payment-intents; GET /customer/payment-attempts/{id}; POST /customer/bills/lookup; POST /customer/bills/{ref}/payment-intents. Use short-lived opaque tokens, rate limits, anti-enumeration, server-side validation and tenant/outlet scoping. Never expose sequential internal IDs.

2.5 Payment safety architecture

The browser initiates a payment intent but does not decide payment truth. Server stores provider, intent ID, expected amount/currency, order/bill reference, idempotency key, expiry and status. Webhook endpoint verifies provider signature, event identity and amount/currency; processes idempotently; stores immutable payment event; reconciles provider status when needed. A customer return page polls server status. A static UPI QR is a merchant identifier only and must use manual merchant-side verification unless a supported merchant integration exists. For cards, use a PCI-compliant hosted checkout/terminal provider; do not collect or store PAN/CVV in the café application.

2.6 WhatsApp and notification architecture

Use WhatsApp Business Platform or approved BSP. Store message template name/version, consent timestamp/source, destination, related order, provider message ID, queued/sent/delivered/failed status and retry count. Send only transactional templates approved for the use case; keep promotional consent separate. Queue messages after durable order creation and after verified payment. Use transactional outbox to avoid losing notifications. Never block order creation on WhatsApp availability.

2.7 3D performance and fallback decisions

Target fast first contentful experience before full scene hydration. Load low-poly optimized glTF assets, compressed textures (KTX2/Basis), meshopt/draco as appropriate, texture atlases, lazy-loaded environment, baked lighting where possible, and adaptive quality based on device capability. Keep draw calls, texture memory and shader complexity bounded. Use Web Workers only where supported and beneficial. Pause animation when tab is hidden. Provide 2D fallback, static poster, and a “Skip intro” control. Respect prefers-reduced-motion; never make motion mandatory to complete checkout.

2.8 ADR additions

ADR-CX-01: immersive customer journey approved over conventional QR storefront; reason is distinctive brand experience and guided order flow. ADR-CX-02: 3D manager is a visual guide, not an autonomous conversational AI; scripted state-driven dialogue first. ADR-CX-03: customer app remains a PWA/mobile web, no mandatory app install. ADR-CX-04: server owns catalog prices, totals, order lifecycle and payment state. ADR-CX-05: graceful 2D fallback is mandatory. ADR-CX-06: use hosted/approved payment provider; no card data storage. ADR-CX-07: WhatsApp is asynchronous and cannot be a prerequisite for order acceptance.

2.9 Security, privacy and abuse controls

Public QR experience requires short-lived session tokens, signed outlet/table context, origin/CORS controls, CSRF protections where cookie sessions are used, rate limiting, bot/spam controls, input sanitization, output encoding, content security policy, dependency scanning, secure headers and audit correlation. Phone lookup must not disclose whether a customer exists. Bill lookup must combine reference plus a verification factor and return only minimal details. Keep PII out of analytics logs and client error telemetry. Define retention/deletion policy before production.

DARK CINEMATIC CAFÉ-TECH | VERSION 1.0 | 06 NOVEMBER 2026